Auditing SHAP Explanations Against Latent Failure Mechanisms in Predictive Maintenance

Koshik Debanath
Independent Researcher
koshik.debanath@gmail.com

Abstract

Analysts use SHAP (SHapley Additive exPlanations) to explain predictive maintenance (PdM) classifiers, but rarely test whether an explanation matches the physical cause of failure. Such a test needs ground truth about why each failure happened, which PdM data rarely provides, and we show that an obvious test for it certifies a model fitted to shuffled labels. We use the AI4I 2020 dataset, whose failure label is the logical OR of separately recorded failure mechanisms. We train a classifier on the combined label only, compute out-of-sample SHAP values by cross-fitting, and test three statistics per mechanism against 1,000 models retrained on permuted labels: the share of attribution on the mechanism's trigger features, how much larger that share is on the mechanism's rows than on other failures, and whether any other feature takes attribution specifically on those rows. With Holm correction, the tuned XGBoost model (test F1 0.89) passes both tests for heat dissipation, power, and overstrain failure in all five seeds (adjusted $p \leq 0.02$), with a second explainer, and with a random forest. Tool wear failure fails with path-dependent TreeSHAP and passes with interventional TreeSHAP, so its verdict depends on the explainer. A synthetic shortcut sensor is flagged once it carries about a fifth to a third of the attribution on the affected mechanism's rows, but not at 13%, while test F1 stays between 0.84 and 0.92. On data generated from the AI4I process at up to 300,000 rows, the relevant-share test stops flagging a shortcut that carries 27% of the attribution, while the third statistic still flags it.

Keywords: explainable AI, SHAP, predictive maintenance, faithfulness evaluation, model randomization, cyber-physical systems

1 Introduction

Predictive maintenance (PdM) systems predict equipment failure from sensor readings of cyber-physical systems, where a wrong reason for a correct alarm can send a technician to the wrong component. Users typically explain these models with post-hoc attributions such as SHAP [25] and read the result as evidence that the model relies on sensible inputs [12, 35]. The explanation itself is rarely tested against the actual cause of failure: a classifier with high accuracy can still rely on a correlate of failure, a shortcut [14, 23], and a plausible SHAP plot does not reveal this.

Testing an explanation against causes requires knowing, for each failure, which mechanism produced it, and industrial datasets seldom record this. A second difficulty is statistical. Rows that belong to a failure mechanism are extreme in that mechanism's features, and we find that a test which ignores this effect passes a model fitted to shuffled labels (Section 6.3).

The AI4I 2020 dataset [27] records the mechanism of each failure. Its target, `Machine failure`, is the logical OR of five flags: tool wear failure (TWF), heat dissipation failure (HDF), power failure

(PWF), overstrain failure (OSF), and random failure (RNF). The first four follow published trigger rules on the sensor variables (Table 1); RNF does not. We train on the combined label only and keep the flags for an audit. For each mechanism, we measure how much of the model's out-of-sample SHAP attribution falls on the features its trigger rule uses, and whether that concentration is specific to its own rows, against models retrained on permuted labels [2].

We test three hypotheses:

- **H1:** for each physical mechanism, the model's attribution concentrates on the trigger features (1) more than permuted-label models do and (2) more on that mechanism's rows than on other failures, and (3) no other feature takes attribution specifically on those rows;
- **H2:** the audit flags a model that relies on a shortcut sensor for the mechanism the shortcut imitates, while mechanisms that passed before keep passing;
- **H3:** a naive null, which compares random failure rows under the same model, does not separate a trained model from a permuted-label one.

H1 holds for HDF, PWF, and OSF; for TWF it fails with the default explainer and holds with interventional TreeSHAP (Fig. 1). H2 holds once the shortcut carries about a third of the attribution, at every data size we test when the third statistic is used; H3 holds.

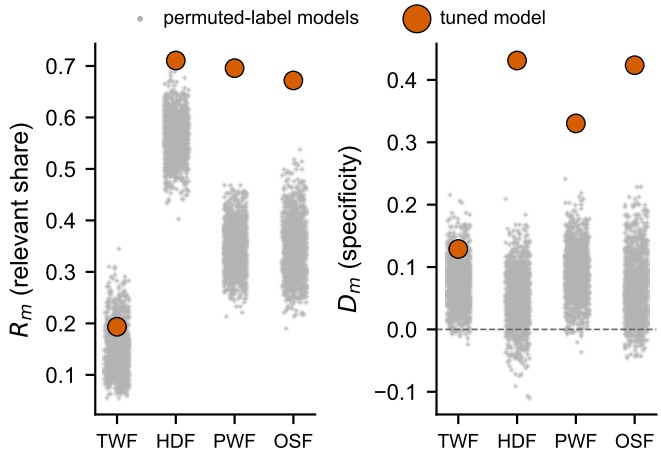


Figure 1: Out-of-sample audit statistics of the tuned model (orange) against 1,000 permuted-label models (gray). R_m (Eq. (2)) is the relevant share and D_m (Eq. (3)) its excess on the mechanism's own rows. HDF, PWF, and OSF lie outside the null on both; TWF lies inside it on both.

Our contributions are:

- a per-mechanism SHAP audit with three statistics, out-of-sample attributions, a model-randomization null, and multiple-comparison correction (Section 4);
- evidence that a naive within-model null is confounded (Section 6.3);
- a graded shortcut experiment that measures when the audit detects reliance on a spurious sensor (Section 6.4), plus seed, single-mechanism, in-sample, raw-feature, explainer, and model-family analyses;

- a scaling study on data generated from the AI4I 2020 process at up to $3 \cdot 10^5$ rows (timed up to 10^6), which shows how the tests' power and cost change with data size (Section 6.6);
- an imbalance-aware benchmark of seven model families on AI4I 2020 with confidence intervals, and a code base that regenerates every number, table, and figure.

The resampling methods, models, explainer, and randomization idea are existing tools; the audit statistics, the null analysis, and the controls are ours.

2 Related Work

Explanations on AI4I 2020. Matzka [27] introduced the dataset with local explanations of individual predictions, and Zhao et al. [38] report global SHAP importance for tree ensembles on it. Neither tests attributions against the mechanism flags or defines a pass/fail criterion, so there is no audit to compare against. For predictive performance, Su and Tseng [34] report random forest positive-class F1 of 0.722 on the binary label with engineered physics-informed features (Table 6). Khemani and Qureshi [20] report LightGBM macro-F1 of 0.814 ± 0.012 over the failure-mode classes from raw features; that task differs from ours, so we do not compare numbers with it.

Evaluating attributions against ground truth. A line of work tests explainers on data where the important features are known by construction: XAI-Bench [24] and OpenXAI [3] generate synthetic tabular data, BAM [36] and Zhou et al. [39] modify images or text so that a known feature drives the label, and CLEVR-XAI [5] provides ground-truth masks for visual questions. ROAR [18] and ROAD [31] avoid ground truth by removing the features an explainer ranks highest and measuring the change in performance. These methods ask whether an explainer reports what the model uses. We ask the complementary question: given TreeSHAP, whose values are exact for tree ensembles [26], does the model use what the data-generating mechanism uses? The benchmarks above control the data-generating process, which AI4I 2020 does not expose beyond its trigger rules, so we do not run them as baselines. Chen et al. [9] distinguish attributions that are "true to the model" from ones "true to the data"; our audit targets the second property at the level of mechanism subgroups.

Randomization tests. Permuted-label retraining is an established null for classifier performance [28] and for feature importance: Altmann et al. [4] retrain on permuted labels to obtain null importances per feature. Adebayo et al. [2] use model randomization to show that some saliency methods give nearly the same explanation for a trained model and a randomized one, and later work questions what such checks can establish [21, 37]. We use the permuted-label null of Altmann et al., but for a different statistic: attribution shares on mechanism subgroups rather than global importance of single features.

Limits of SHAP. Shapley-value attributions depend on how absent features are modeled [19, 22]; with dependent features, as our derived features are, the observational and interventional variants can differ [1], and explanations can be manipulated [33]. We therefore repeat the audit with interventional TreeSHAP (Section 6.5).

Causal PdM. Dhande et al. [13] compare a Bayesian structural causal model with correlational models on 10,000 computer numerical control (CNC) records with 3.3% failures, which matches AI4I 2020, and report that the causal model attributes HDF, PWF, and OSF failures correctly. They replace the correlational model with a causal one; we keep the kind of correlational model that dominates PdM work [32] and test its explanation.

3 Background and Problem Setting

SHAP. For a model f and input x with d features, SHAP assigns each feature j an attribution $\phi_j(x)$, its Shapley value with respect to the model output, such that $f(x) = \phi_0 + \sum_{j=1}^{d} \phi_j(x)$, where ϕ_0 is the expected output [25]. TreeSHAP computes these values exactly for tree ensembles [26]; by default it follows the tree paths (path-dependent), and an interventional variant averages over a background sample instead. We use $|\phi_j(x)|$ as the magnitude of feature j's contribution to the prediction for x.

Problem setting. We observe rows (x_i, y_i, z_i), where x_i is the sensor vector, $y_i \in \{0, 1\}$ the failure label, and $z_i \in \{0, 1\}^M$ holds M mechanism flags. We train f on (x_i, y_i) only. For each mechanism m, domain rules give a set E_m of features the mechanism's trigger depends on. With $S_m = \{i : z_{i,m} = 1\}$ and $F = \{i : y_i = 1\}$, the audit asks whether attributions on S_m concentrate on E_m, and more so than on $F \setminus S_m$. Two assumptions are unusual: E_m must be known, and the flags must exist for some failures even though the model never sees them.

4 Audit Method

For a set of rows S and mechanism m, let $I_{m,j}(S) = |S|^{-1} \sum_{i \in S} |\phi_j(x_i)|$ and define the share of attribution on the trigger features,

$$r_m(S) = \sum_{j \in E_m} I_{m,j}(S) \Big/ \sum_{j=1}^{d} I_{m,j}(S). \tag{1}$$

The audit uses two statistics, the *relevant share* and the *specificity*,

$$R_m = r_m(S_m), \tag{2}$$

$$D_m = r_m(S_m) - r_m(F \setminus S_m). \tag{3}$$

R_m is large when attribution on the mechanism's rows falls on its trigger features; D_m is large when this concentration is specific to those rows rather than common to all failures.

Both statistics can be large while a feature outside E_m also takes a sizable share, so we add a converse statistic, the *shortcut score*. With $s_j(S) = I_{m,j}(S) / \sum_k I_{m,k}(S)$ the share of feature j,

$$S_m = \max_{j \notin E_m} \big[s_j(S_m) - s_j(F \setminus S_m) \big]. \tag{4}$$

S_m is large when some feature outside the trigger set takes attribution specifically on the mechanism's rows, which is what a shortcut for that mechanism does. A mechanism passes the audit if R_m and D_m are significantly large and S_m is not; the first two establish that the model uses the mechanism's features, the third that it does not also rely on another feature for it. S_m is related to conditional randomization and knockoff tests [7], which ask whether a feature matters given the others, but it is computed from the explanation and restricted to the mechanism's rows.

Out-of-sample attributions. Attributions on training rows reflect what the model fitted, not what it generalizes. We therefore cross-fit [11]: we split the data into five stratified folds, train the full pipeline on four, and compute SHAP values for the held-out fold, so that every audited row is explained by a model that never saw it.

Naive null. An obvious test compares R_m with the same statistic on random, equally sized subsets of F under the same model. It passes a model fitted to shuffled labels (Section 6.3). A likely reason is that rows in S_m are extreme in the features of E_m, because the trigger rule selects

them by those features, and $|\phi_j|$ tends to be larger at atypical feature values whatever the model has fitted.

Model-randomization null. We rerun the full procedure – features, resampling, fitting, and cross-fitting – K times on permuted labels and compute the three statistics for each run. The hyperparameters stay at the values tuned on the real labels, so the null runs differ from the audited model only in the labels. For $T \in \{R_m, D_m, S_m\}$ with null values $T^{(k)}$,

$$p = \frac{1 + \#\{k : T^{(k)} \geq T\}}{K + 1}, \tag{5}$$

and we adjust the four physical mechanisms for multiple comparisons with Holm's method [17], separately for each statistic. R_m and D_m pass if their adjusted p is below 0.05; S_m flags a shortcut if its adjusted p is below 0.05.

Random failures. RNF has no trigger features. We report the share of its single largest feature and whether it is more concentrated than under the null, as a descriptive check rather than a pass/fail test.

5 Experimental Setup

Dataset. AI4I 2020 has 10,000 rows: a quality variant `Type` $\in \{L, M, H\}$, five process variables (air and process temperature, rotational speed, torque, tool wear), the failure label, and the five mechanism flags. 339 rows (3.39%) are failures. The labels depart from the stated OR rule: 24 rows carry several mechanism flags, 9 failures carry none, and 18 of the 19 RNF rows have `Machine failure` $= 0$, which is why we treat RNF descriptively. We audit all flagged rows and, separately, rows with exactly one flag.

Table 1: Failure mechanisms in AI4I 2020 [27] and their trigger features E_m; counts as released. The TWF failure point is random within the window.

Mech.	Trigger	E_m	n
TWF	tool wear at a random point in 200–240 min	tool wear	46
HDF	(process − air temp.) < 8.6 K and speed < 1380 rpm	both temps., their difference, speed	115
PWF	torque × speed (rad/s) outside [3500, 9000] W	torque, speed, power	95
OSF	tool wear × torque above 11,000/12,000/13,000 min·Nm for L/M/H	tool wear, torque, overstrain, its ratio	98
RNF	0.1% chance, independent of the process	–	19

Features. We derive four features from the trigger rules without using the flags: *power* (torque × speed), *temperature difference* (process − air), *overstrain* (tool wear × torque), and *overstrain ratio* (overstrain over the type-specific limit). The mechanism flags are components of the label, so we exclude them from the features.

Training. We split the data 70/20/10 into training, validation, and test sets, stratified by label, and fit scaling and one-hot encoding on the training set. We compare three oversamplers – the synthetic minority oversampling technique (SMOTE) [8], Borderline-SMOTE [15], and ADASYN [16]

– by the validation F1 of a default XGBoost [10] model and use the best, Borderline-SMOTE (Table 9). We train seven model families: logistic regression, a support vector machine (SVM), k-nearest neighbors (k-NN), a decision tree, a random forest, XGBoost, and a multilayer perceptron (MLP) (Table 2; the tuned XGBoost is the eighth row, not a separate family).

Table 2: Model settings. Unlisted hyperparameters are library defaults (scikit-learn 1.9.1, XGBoost 3.4.1, PyTorch 2.14.0); random seed 42.

Model	Settings
Logistic regression	max. 1000 iterations
SVM	radial basis function (RBF) kernel, probability outputs
k-NN	$k = 7$
Decision tree	max. depth 10
Random forest	300 trees, max. depth 12
XGBoost	300 trees, max. depth 6, learning rate 0.1
XGBoost (tuned)	200 trees, max. depth 8, learning rate 0.1, subsample 0.8, column sample 0.8, min. child weight 1, $\gamma = 0$
MLP	64 and 32 hidden units, ReLU, batch norm, dropout 0.3/0.2; Adam (learning rate 10^{-3}, weight decay 10^{-5}), binary cross-entropy, batch 64, 40 epochs

Selection and tuning. We select on the validation set only: XGBoost has the highest validation F1, so we tune it with a 40-iteration randomized search under 5-fold stratified cross-validation, scored on F1, with Borderline-SMOTE inside the pipeline so that each fold resamples only its own training part, over depth 3–8, 200–600 trees, learning rate 0.01–0.2, subsample and column sample 0.6–1.0, minimum child weight 1–5, and γ 0–0.3. We set the decision threshold to maximize F1 on the validation precision-recall (PR) curve. Confidence intervals (CIs) are exact binomial (Clopper–Pearson) for precision and recall and a 5,000-sample stratified bootstrap otherwise. We also run 5-fold stratified cross-validation of the whole pipeline at the default threshold of 0.5. The hyperparameters are tuned on the training split, which the cross-validation and the audit reuse, so the cross-validated F1 is mildly optimistic.

Audit. The primary audit cross-fits the tuned pipeline with seed 42 and $K = 1{,}000$ permuted-label runs, so the smallest attainable p before correction is 0.001. We repeat it with seeds 1–4 ($K = 200$), with raw features only ($K = 1{,}000$; without the four derived features, to test whether they make the audit circular), and with SHAP from a single model fitted on all rows and explaining those same rows (*in-sample*, $K = 1{,}000$). Each cross-fitted run costs $5(K + 1)$ model fits; the runs are independent, and the primary audit takes 12 minutes on eight CPU cores.

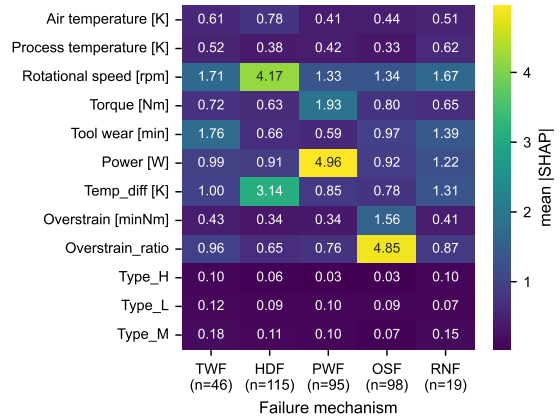
Explainer and model variants. We repeat the primary audit ($K = 200$) with interventional TreeSHAP, using backgrounds of 100, 500, and 1,000 training rows per fold (three background seeds for 100 and 500), and with the random forest of Table 2 in place of XGBoost; the null runs use the same explainer and model. The interventional values sum to the model's margin output to within $5 \cdot 10^{-6}$.

Scaling. To study larger data, we generate new data from the process documented with AI4I 2020 [27], copying no released row: quality type L/M/H with probability 0.6/0.3/0.1; air temperature $\mathcal{N}(300, 2)$ K; a process–air difference $\mathcal{N}(10, 1)$ K correlated -0.70 with air temperature, as in the release; torque $\mathcal{N}(40, 10)$ Nm; speed as a quadratic in inverse torque fitted to the

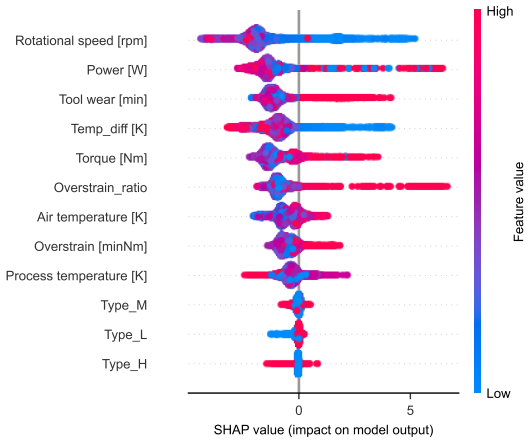
release plus residual noise; and tool wear that grows by 2/3/5 min per row for L/M/H until the tool reaches a life drawn from $U(200, 240)$ min, where it fails (TWF) with probability 0.4 or is replaced. The documented temperatures are random walks; we draw rows independently, because the model never sees row order and a slowly varying walk makes the HDF rate of 10^4 rows depend on a few excursions. HDF, PWF, and OSF follow the published rules, which reproduce the released flags exactly, and RNF fires with probability 0.001. The generated failure rate is 3.5–4.0%, against 3.4% released. We run the audit at $n \in \{10^4, 3 \cdot 10^4, 10^5\}$ with three generation seeds and at $3 \cdot 10^5$ with one, each without the shortcut sensor and with it at $\sigma \in \{0.35, 0.25\}$ ($K = 200$, smallest adjusted p 0.020), and we time one cross-fitted run up to 10^6 rows.

Controls. A model trained on permuted labels is exchangeable with the null runs, so its p-values are uniform by construction; we use it only as a sanity check of the code and as the case that the naive null must fail. For H2, we add a synthetic "sensor" $s = z_{\text{HDF}} + \mathcal{N}(0, \sigma^2)$ with no physical role and vary σ from 2 to 0.1. For each σ we report the sensor's share of attribution on HDF rows, test F1, and the audit verdicts ($K = 500$). The sensor is not in E_{HDF}, so a model that relies on it should fail the HDF audit.

6 Results and Discussion



(a) Mean out-of-sample $|\phi_j|$ on the rows of each mechanism



(b) Global SHAP summary on all rows

Figure 2: SHAP attributions of the tuned model. The audit tests the per-mechanism view (a); the global view (b) ranks features overall but does not separate the mechanisms.

6.1 Predictive performance

Table 5 lists test metrics. Tree ensembles outperform the linear and instance-based models. The tuned model at its validation-selected threshold of 0.978 reaches F1 0.89, against 0.87 for the default XGBoost at 0.5; the confidence intervals overlap, so neither tuning nor the threshold is shown to help. The test set holds 34 failures, so the uncertainty is large: the tuned model flags 27 rows, all true failures, which gives precision 1.00 (95% CI 0.87–1.00) and recall 0.79 (0.62–0.91). Its scores are calibrated (Brier score 0.0090 against 0.033 for a constant prediction at the base rate; expected calibration error 0.008). Five-fold cross-validation of the full pipeline gives positive-class

Table 3: Primary audit (cross-fitted, seed 42, $K = 1{,}000$). Null: mean over permuted-label runs. p: Holm-adjusted over the four physical mechanisms; $^{\times}$ marks a failed R_m or D_m test ($p \geq 0.05$); S_m (Eq. (4)) flags a shortcut if $p < 0.05$. For RNF the statistic is the largest feature's share and p tests for more concentration than the null.

	n	Top feature	R_m	Null	p	D_m	Null	p	S_m	Null	p
TWF	46	tool wear	0.193	0.152	$0.201^{\times}$	0.129	0.070	$0.077^{\times}$	0.025	0.025	1.000
HDF	115	speed	0.711	0.557	0.004	0.431	0.049	0.004	0.001	0.015	1.000
PWF	95	power	0.696	0.347	0.004	0.330	0.088	0.004	0.001	0.011	1.000
OSF	98	overstrain ratio	0.672	0.345	0.004	0.424	0.067	0.004	-0.001	0.019	1.000
RNF	19	speed	0.186	0.240	0.985	–	–	–	–	–	–

Table 4: Holm-adjusted audit p-values for R_m, D_m, and S_m across analysis variants; $^{\times}$ marks a failed R or D test ($p \geq 0.05$), † a flagged shortcut ($p_S < 0.05$); no variant flags one. Interventional rows use background seed 0; seeds 1 and 2 give the same verdicts. The smallest attainable adjusted p is $4/(K+1)$. The permuted-label model is exchangeable with its null and serves only as a sanity check; the last two rows apply the naive null of Section 4. The interventional and random forest rows are discussed in Section 6.5.

		TWF			HDF			PWF			OSF		
Variant	K	R	D	S	R	D	S	R	D	S	R	D	S
Primary, seed 42	1000	$0.201^{\times}$	$0.077^{\times}$	1.000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000
single-mechanism rows	1000	$0.187^{\times}$	$0.076^{\times}$	1.000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000
Seed 1	200	$0.199^{\times}$	$0.119^{\times}$	0.776	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Seed 2	200	$0.149^{\times}$	$0.065^{\times}$	0.498	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Seed 3	200	$0.174^{\times}$	$0.095^{\times}$	0.736	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Seed 4	200	$0.269^{\times}$	$0.129^{\times}$	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Raw features only	1000	$0.345^{\times}$	$0.616^{\times}$	1.000	$0.190^{\times}$	0.004	1.000	0.004	0.004	1.000	0.004	0.048	1.000
In-sample SHAP	1000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000	0.004	0.004	1.000
Interventional, background 100	200	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Interventional, background 500	200	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Interventional, background 1000	200	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.020	0.020	1.000
Random forest	200	$0.617^{\times}$	$0.398^{\times}$	1.000	0.020	0.020	1.000	0.020	0.020	1.000	0.040	0.020	1.000
Permuted labels (sanity)	200	$1.000^{\times}$	$0.731^{\times}$	1.000	$0.199^{\times}$	$0.557^{\times}$	1.000	$1.000^{\times}$	$0.731^{\times}$	1.000	$1.000^{\times}$	$0.731^{\times}$	0.637
Naive null, tuned model	–	0.002	–	–	0.002	–	–	0.002	–	–	0.002	–	–
Naive null, permuted labels	–	0.002	–	–	0.002	–	–	0.002	–	–	0.002	–	–

F1 0.822±0.013 and macro-F1 0.908±0.007, close to the 0.831 of the tuning search, which resamples inside its folds (Table 6). Figs. 3 and 4 show PR curves and the confusion matrix.

Table 5: Positive-class test metrics, sorted by F1; bootstrap 95% CI for F1. ROC-AUC: area under the receiver operating characteristic curve.

Model	Prec.	Rec.	F1 (95% CI)	ROC-AUC
XGBoost (tuned)	**1.00**	0.79	**0.89** (0.79–0.95)	0.988
XGBoost	0.86	0.88	0.87 (0.78–0.94)	0.988
Random forest	0.79	0.88	0.83 (0.74–0.92)	**0.989**
MLP	0.65	**0.91**	0.76 (0.67–0.84)	0.988
Decision tree	0.68	0.82	0.75 (0.64–0.84)	0.927
SVM	0.42	0.88	0.57 (0.48–0.65)	0.976
k-NN	0.43	0.79	0.56 (0.46–0.65)	0.877
Logistic regression	0.24	0.85	0.37 (0.31–0.43)	0.943

Table 6: Comparison with prior results on AI4I 2020.

Study and protocol	Metric	Value
Random forest, single split [34]	pos.-class F1	0.722
Ours, single split	pos.-class F1	0.89
Ours, 5-fold CV	pos.-class F1	0.822 ± 0.013

6.2 Audit of the tuned model (H1)

For HDF, PWF, and OSF the tuned model passes the relevant-share and specificity tests, and the shortcut score flags no mechanism (Table 3, Fig. 1). On each mechanism's held-out rows, the share of attribution on its trigger features exceeds that of every permuted-label run, and it exceeds the share on other failures by 0.33 to 0.43, against null means below 0.09. The highest-attribution feature is a trigger feature in each case. Fig. 2a shows the attribution matrix.

TWF fails both tests ($p = 0.201$ and 0.077). Tool wear is its highest-attribution feature, but it takes only 19% of the attribution on TWF rows, against 15% under the null. The TWF trigger fires at a random point within a tool-wear window, so tool wear only partly determines it, and the 46 TWF rows give little to generalize from.

6.3 Robustness (H1, H3)

Table 4 repeats the audit. The HDF, PWF, and OSF verdicts hold in all five seeds and on single-mechanism rows. TWF fails both tests in every seed.

With SHAP from a model that has seen every audited row, all four mechanisms pass, including TWF ($p = 0.004$; $R_{\mathrm{TWF}} = 0.35$ against 0.19 out-of-sample). The difference from the cross-fitted result shows that in-sample attributions can overstate how well a model's explanation matches a mechanism.

Without the derived features, PWF and OSF pass both tests (OSF specificity narrowly, $p = 0.048$), HDF passes only on specificity, and TWF on neither. The derived features therefore help the HDF verdict but are not needed for PWF and OSF: the audit is not circular for those two, and a mechanism defined by a difference of inputs, as HDF is, is harder to audit when the model must approximate the difference with splits.

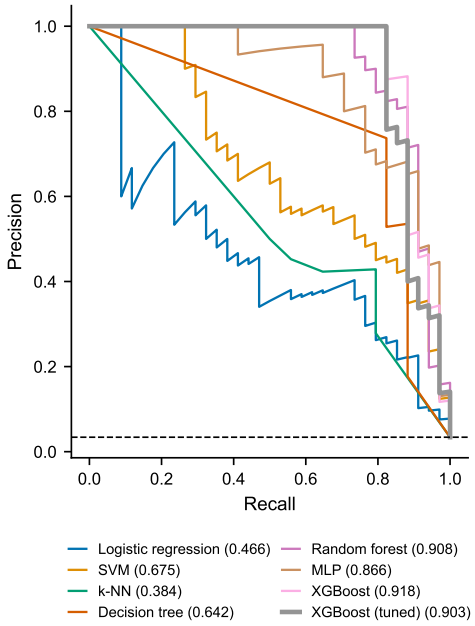


Figure 3: Test PR curves (average precision in parentheses); the dashed line is the 3.4% failure rate.

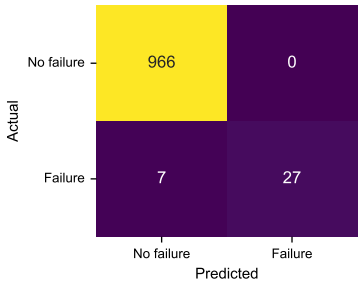


Figure 4: Confusion matrix of the tuned model on the test set.

The permuted-label model fails every test under the model-randomization null, as it must. The naive null passes it on all four mechanisms (Holm-adjusted $p = 0.002$), exactly as it passes the tuned model, including on TWF, which the model-randomization null rejects. The naive null therefore does not separate the two models, which supports H3.

6.4 Shortcut sweep (H2)

Table 7: Shortcut sweep ($K = 500$): test F1, the sensor's share of attribution on HDF rows, R_{HDF}, Holm-adjusted p-values for HDF and TWF, and the verdict for PWF and OSF. $^{\times}$ marks a failed R or D test ($p \geq 0.05$), † a flagged shortcut ($p_S < 0.05$).

			HDF				TWF	
σ	F1	Sensor	R	p_R	p_D	p_S	p_R / p_D	PWF, OSF
2.0	0.90	0.025	0.705	0.008	0.008	1.000	$0.134^{\times}$ / $0.052^{\times}$	pass
1.0	0.89	0.023	0.695	0.008	0.008	1.000	$0.102^{\times}$ / 0.044	pass
0.67	0.88	0.033	0.678	0.008	0.008	1.000	$0.130^{\times}$ / $0.058^{\times}$	pass
0.5	0.85	0.051	0.656	0.008	0.008	1.000	$0.138^{\times}$ / $0.062^{\times}$	pass
0.4	0.89	0.080	0.621	0.008	0.008	1.000	$0.136^{\times}$ / $0.056^{\times}$	pass
0.35	0.92	0.126	0.574	0.044	0.008	1.000	$0.128^{\times}$ / 0.0499	pass
0.3	0.90	0.209	0.475	$0.365^{\times}$	0.008	0.407	$0.224^{\times}$ / 0.030	pass
0.25	0.87	0.350	0.308	$0.996^{\times}$	0.012	$0.008^{\dagger}$	$0.228^{\times}$ / 0.048	pass
0.2	0.90	0.425	0.222	$1.000^{\times}$	$0.084^{\times}$	$0.008^{\dagger}$	$0.248^{\times}$ / $0.084^{\times}$	pass
0.15	0.84	0.456	0.185	$1.000^{\times}$	$0.168^{\times}$	$0.008^{\dagger}$	$0.279^{\times}$ / $0.168^{\times}$	pass
0.1	0.87	0.405	0.218	$1.000^{\times}$	$0.068^{\times}$	$0.008^{\dagger}$	$0.232^{\times}$ / $0.074^{\times}$	pass

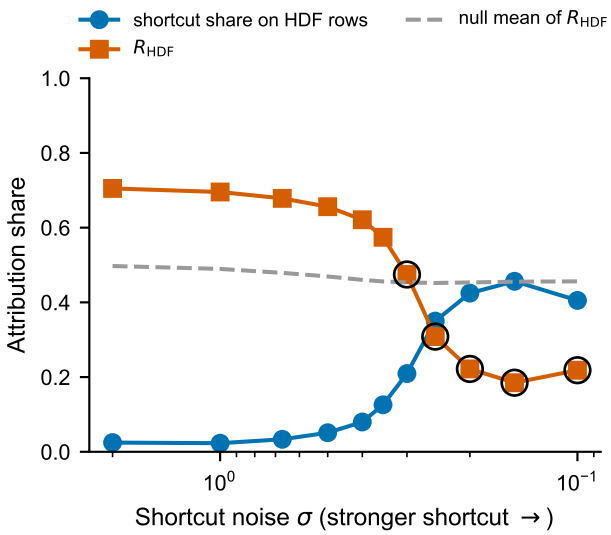


Figure 5: Shortcut sweep. Circles mark noise levels at which the HDF relevant-share test fails.

Table 7 and Fig. 5 show the shortcut sweep. For $\sigma \geq 0.4$ the sensor receives at most 8% of the attribution on HDF rows and the audit passes, which is correct because the model barely uses it. At $\sigma = 0.35$ the sensor takes 13% and the relevant-share test passes narrowly ($p = 0.044$). From $\sigma = 0.3$, where the sensor takes 21%, the HDF relevant-share test fails; from $\sigma = 0.25$ (35%) the shortcut score flags the sensor as well, and from $\sigma = 0.2$ (42%) the specificity test fails. Test F1 stays between 0.84 and 0.92 across the sweep, so accuracy does not reveal the shortcut. On

the released data H2 therefore holds once a shortcut takes about a fifth of the attribution, not for lighter reliance. PWF and OSF pass at every noise level. TWF fails the relevant-share test at every level; its specificity test passes narrowly at four levels ($0.030 \leq p \leq 0.0499$) and fails at the other seven, so TWF stays near the decision boundary.

The shortcut score is conservative here because the null runs see the sensor too: a permuted-label model gives the sensor attribution on HDF rows, where its values are atypical, so the null mean of S_{HDF} rises to 0.09. This is the confound of the naive null (Section 4), and the model-randomization null absorbs it.

6.5 Explainer and model variants

The HDF, PWF, and OSF verdicts hold with interventional TreeSHAP and with a random forest (Table 4). TWF does not behave consistently: it passes both tests with interventional TreeSHAP at every background size and seed (R_{TWF} between 0.45 and 0.47, tool wear its top feature) and fails both with the random forest ($R_{\text{TWF}} = 0.13$, speed its top feature). Tool wear has Spearman correlation 0.92 with both overstrain features, and path-dependent TreeSHAP spreads credit over features correlated with those a tree splits on, whereas the interventional variant breaks that dependence [1]. An audit verdict is therefore a statement about a model *and* an explainer, and a mechanism that passes under one explainer only should be reported as unresolved.

6.6 Scaling (H1, H2)

Table 8: Audit on data generated from the AI4I 2020 process at size n ($K = 200$): mean sensor share of the HDF attribution, and the fraction of generation seeds in which the HDF shortcut is flagged by the relevant-share test ($p_R \geq 0.05$) or the shortcut score ($p_S < 0.05$), TWF passes the specificity test, or the shortcut score flags TWF. Min.: wall-clock minutes for one audit on eight cores. HDF, PWF, and OSF pass the R and D tests in every run without the sensor.

				HDF flagged by		TWF		
n	σ	Seeds	Sensor	R	S	D pass	S flag	Min.
10,000	–	3	0.000	0/3	0/3	0/3	0/3	3
	0.35	3	0.158	2/3	0/3	0/3	0/3	3
	0.25	3	0.390	3/3	3/3	0/3	0/3	3
30,000	–	3	0.000	0/3	0/3	0/3	1/3	7
	0.35	3	0.128	0/3	0/3	2/3	0/3	6
	0.25	3	0.346	3/3	3/3	2/3	1/3	7
100,000	–	3	0.000	0/3	0/3	1/3	2/3	17
	0.35	3	0.100	0/3	0/3	2/3	1/3	19
	0.25	3	0.315	3/3	3/3	2/3	1/3	19
300,000	–	1	0.000	0/1	0/1	0/1	0/1	52
	0.35	1	0.063	0/1	0/1	0/1	0/1	59
	0.25	1	0.271	0/1	1/1	0/1	1/1	61

Without the sensor, HDF, PWF, and OSF pass the relevant-share and specificity tests and are never flagged by the shortcut score, at every size and seed (Table 8). TWF fails the relevant-share test in every run; its specificity test passes in one of the seven runs from 30,000 rows and in none at 10,000, so more data does not resolve TWF. The shortcut score flags TWF in 3 of the 10 runs without the sensor, each time for process temperature, which plays no role in tool wear failure; the effect is small ($S_{\text{TWF}} \leq 0.05$) but indicates that the model uses a non-trigger feature on TWF rows.

The two shortcut tests diverge as n grows. At $\sigma = 0.25$ the sensor takes 27–45% of the HDF attribution. The relevant-share test flags it in all nine runs up to 100,000 rows but not at 300,000, where $R_{\text{HDF}} = 0.472$ still exceeds the 95th null percentile of 0.417: with more data the null tightens, and "more than a model that learned nothing" becomes a low bar. The shortcut score flags the sensor in all ten runs, including at 300,000 rows ($S_{\text{HDF}} = 0.249$ against a 95th percentile of 0.211). Neither test flags the lighter shortcut at $\sigma = 0.35$ (6–20% of the attribution) except the relevant-share test in two runs at 10,000 rows. H2 therefore holds at every size for a shortcut that takes about a third of the attribution, provided the audit includes the shortcut score, and does not hold for a shortcut of a tenth.

The audit costs $5(K+1)$ model fits. One cross-fitted run takes 1.3 s at 10^4 rows, 31 s at $3 \cdot 10^5$, and 115 s at 10^6 (38,230 failures) on one core, roughly linear in n. On eight cores an audit with $K = 200$ took 3 minutes at 10^4 rows and about an hour at $3 \cdot 10^5$, four times the ideal because parallel runs compete for memory bandwidth; at 10^6 rows and $K = 1{,}000$ the ideal is 4 hours on eight cores and, with the same slowdown, about 16. The runs are independent, so the cost divides across machines.

6.7 Resampling

Table 9: Oversampler comparison on the validation set (default XGBoost). PR-AUC: area under the precision-recall curve.

Sampler	Prec.	Rec.	F1	PR-AUC
Borderline-SMOTE	**0.814**	0.838	**0.826**	0.883
SMOTE	0.699	**0.853**	0.768	**0.892**
ADASYN	0.683	0.824	0.747	0.876

Borderline-SMOTE has the highest validation F1 (Table 9), with the other two oversamplers trading precision for recall.

6.8 Limitations

AI4I 2020 is simulated from the same rules we use as ground truth, so the audit checks agreement with the simulator's logic, not with independent measurements of real machines; we contribute a protocol, not a finding about real equipment. Autran et al. [6] show that realistic irregularities added to AI4I 2020 change model performance, so results on the clean data may not transfer.

The audit detects a shortcut that takes a fifth to a third of the attribution but not one that takes a tenth; without the shortcut score, the relevant-share test alone misses even a quarter on large data. We test one kind of shortcut, a noisy copy of one mechanism's flag. The audit needs the trigger features E_m to be known, the derived features influence the HDF verdict, and the TWF verdict depends on the explainer. We audit two model families and two explainers on one simulator, released at 10,000 rows and generated at up to $3 \cdot 10^5$; the generator follows the documented process but draws rows independently. Mechanisms with few rows, such as TWF and RNF, give weak evidence either way.

7 Conclusion

We audited SHAP explanations of a PdM classifier against failure mechanisms that the model never saw. With out-of-sample attributions, a model-randomization null, and Holm correction, the

tuned model's explanations match the heat dissipation, power, and overstrain mechanisms in all five seeds, with two explainers and two model families; the tool wear verdict depends on the explainer. The audit flags a shortcut sensor once it carries a fifth to a third of the attribution, while accuracy stays unchanged. On larger generated data the relevant-share test alone stops flagging such a shortcut, and a converse statistic – whether a non-trigger feature takes attribution specifically on the mechanism's rows – is needed to keep detecting it. A naive null that compares random failure rows under the same model passes a permuted-label model, so subgroup attribution tests need a null that accounts for how atypical the audited rows are.

The audit applies wherever maintenance records log a root cause for failures that a model predicts as a binary risk; we have not measured how common such records are. Next steps are real maintenance logs with recorded root causes, process data with labeled fault types such as the extended Tennessee Eastman simulation [29, 30], shortcuts other than noisy label copies, and a formal power analysis of the detection threshold.

Acknowledgment

Portions of the manuscript text and the analysis code were drafted with the assistance of an AI system (Claude, Anthropic). The author reviewed and verified all content and takes full responsibility for it. Code, model artifacts, and scripts that regenerate every number, table, and figure are available at `https://github.com/kowshik24/glowing-broccoli`.

References

[1] Kjersti Aas, Martin Jullum, and Anders Løland. Explaining individual predictions when features are dependent: More accurate approximations to Shapley values. *Artificial Intelligence*, 298:103502, 2021. doi: 10.1016/j.artint.2021.103502.

[2] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt, and Been Kim. Sanity checks for saliency maps. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[3] Chirag Agarwal, Satyapriya Krishna, Eshika Saxena, Martin Pawelczyk, Nari Johnson, Isha Puri, Marinka Zitnik, and Himabindu Lakkaraju. OpenXAI: Towards a transparent evaluation of model explanations. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, 2022.

[4] André Altmann, Laura Tołoşi, Oliver Sander, and Thomas Lengauer. Permutation importance: A corrected feature importance measure. *Bioinformatics*, 26(10):1340–1347, 2010. doi: 10.1093/bioinformatics/btq134.

[5] Leila Arras, Ahmed Osman, and Wojciech Samek. CLEVR-XAI: A benchmark dataset for the ground truth evaluation of neural network explanations. *Information Fusion*, 81:14–40, 2022. doi: 10.1016/j.inffus.2021.11.008.

[6] J.-V. Autran, V. Kuhn, J.-P. Diguet, M. Dubois, and C. Buche. AI4I-PMDI: Predictive maintenance datasets with complex industrial settings' irregularities. *Procedia Computer Science*, 246:1201–1209, 2024. doi: 10.1016/j.procs.2024.09.546.

[7] Emmanuel Candès, Yingying Fan, Lucas Janson, and Jinchi Lv. Panning for gold: 'model-X' knockoffs for high dimensional controlled variable selection. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 80(3):551–577, 2018. doi: 10.1111/rssb.12265.

[8] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16: 321–357, 2002.

[9] Hugh Chen, Joseph D. Janizek, Scott Lundberg, and Su-In Lee. True to the model or true to the data? *arXiv preprint arXiv:2006.16234*, 2020.

[10] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[11] Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018. doi: 10.1111/ectj.12097.

[12] Logan Cummins, Alexander Sommers, Somayeh Bakhtiari Ramezani, Sudip Mittal, Joseph Jabour, Maria Seale, and Shahram Rahimi. Explainable predictive maintenance: A survey of current methods, challenges and opportunities. *IEEE Access*, 12:57574–57602, 2024. doi: 10.1109/ACCESS.2024.3391130.

[13] Shaunak Dhande, Chutian Ma, Giacinto Paolo Saggese, Paul Smith, and Krishna Taduri. A benchmark of causal vs. correlation AI for predictive maintenance. arXiv:2512.01149, 2026.

[14] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2(11):665–673, 2020. doi: 10.1038/s42256-020-00257-z.

[15] Hui Han, Wen-Yuan Wang, and Bing-Huan Mao. Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning. In *International Conference on Intelligent Computing*, pages 878–887. Springer, 2005.

[16] Haibo He, Yang Bai, Edwardo A. Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *2008 IEEE International Joint Conference on Neural Networks*, pages 1322–1328. IEEE, 2008.

[17] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979. URL `https://www.jstor.org/stable/4615733`.

[18] Sara Hooker, Dumitru Erhan, Pieter-Jan Kindermans, and Been Kim. A benchmark for interpretability methods in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 32, pages 9737–9748, 2019.

[19] Dominik Janzing, Lenon Minorics, and Patrick Blöbaum. Feature relevance quantification in explainable AI: A causal problem. In Silvia Chiappa and Roberto Calandra, editors, *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, volume 108 of *Proceedings of Machine Learning Research*, pages 2907–2916. PMLR, 2020.

[20] Kushal Khemani and Anjum Nazir Qureshi. AI-driven predictive maintenance for connected vehicles using environmental context integration evaluated through simulation benchmarking and field validation. *Discover Vehicles*, 2:19, 2026. doi: 10.1007/s44465-026-00023-2.

[21] Pieter-Jan Kindermans, Sara Hooker, Julius Adebayo, Maximilian Alber, Kristof T. Schütt, Sven Dähne, Dumitru Erhan, and Been Kim. The (un)reliability of saliency methods. In Wojciech Samek, Grégoire Montavon, Andrea Vedaldi, Lars Kai Hansen, and Klaus-Robert Müller, editors, *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*, volume 11700 of *Lecture Notes in Computer Science*, pages 267–280. Springer, 2019. doi: 10.1007/978-3-030-28954-6_14.

[22] I. Elizabeth Kumar, Suresh Venkatasubramanian, Carlos Scheidegger, and Sorelle Friedler. Problems with Shapley-value-based explanations as feature importance measures. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5491–5500. PMLR, 2020.

[23] Sebastian Lapuschkin, Stephan Wäldchen, Alexander Binder, Grégoire Montavon, Wojciech Samek, and Klaus-Robert Müller. Unmasking Clever Hans predictors and assessing what machines really learn. *Nature Communications*, 10(1):1096, 2019. doi: 10.1038/s41467-019-08987-4.

[24] Yang Liu, Sujay Khandagale, Colin White, and Willie Neiswanger. Synthetic benchmarks for scientific research in explainable machine learning. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

[25] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[26] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020. doi: 10.1038/s42256-019-0138-9.

[27] Stephan Matzka. Explainable artificial intelligence for predictive maintenance applications. In *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)*, pages 69–74. IEEE, 2020. doi: 10.1109/AI4I49448.2020.00023.

[28] Markus Ojala and Gemma C. Garriga. Permutation tests for studying classifier performance. *Journal of Machine Learning Research*, 11(62):1833–1863, 2010. URL `https://www.jmlr.org/papers/v11/ojala10a.html`.

[29] Cory A. Rieth, Ben D. Amsel, Randy Tran, and Maia B. Cook. Additional Tennessee Eastman process simulation data for anomaly detection evaluation. Harvard Dataverse, V1, 2017.

[30] Cory A. Rieth, Ben D. Amsel, Randy Tran, and Maia B. Cook. Issues and advances in anomaly detection evaluation for joint human-automated systems. In Jessie Chen, editor, *Advances in Human Factors in Robots and Unmanned Systems*, volume 595 of *Advances in Intelligent Systems and Computing*, pages 52–63. Springer, 2018. doi: 10.1007/978-3-319-60384-1_6.

[31] Yao Rong, Tobias Leemann, Vadim Borisov, Gjergji Kasneci, and Enkelejda Kasneci. A consistent and efficient evaluation strategy for attribution methods. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18770–18795. PMLR, 2022.

[32] Oscar Serradilla, Ekhi Zugasti, Jon Rodriguez, and Urko Zurutuza. Deep learning models for predictive maintenance: A survey, comparison, challenges and prospects. *Applied Intelligence*, 52(10):10934–10964, 2022. doi: 10.1007/s10489-021-03004-y.

[33] Dylan Slack, Sophie Hilgard, Emily Jia, Sameer Singh, and Himabindu Lakkaraju. Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, pages 180–186. ACM, 2020. doi: 10.1145/3375627.3375830.

[34] Yi-Kai Su and Chun-Jan Tseng. A physics-informed benchmarking framework for machine learning and tree-based ensembles in IIoT-enabled predictive maintenance. *Sensors*, 26(16): 5026, 2026. doi: 10.3390/s26165026.

[35] Simon Vollert, Martin Atzmueller, and Andreas Theissler. Interpretable machine learning: A brief survey from the predictive maintenance perspective. In *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–8. IEEE, 2021. doi: 10.1109/ETFA45728.2021.9613467.

[36] Mengjiao Yang and Been Kim. Benchmarking attribution methods with relative feature importance. *arXiv preprint arXiv:1907.09701*, 2019.

[37] Gal Yona and Daniel Greenfeld. Revisiting sanity checks for saliency maps. In *NeurIPS 2021 Workshop on eXplainable AI Approaches for Debugging and Diagnosis (XAI4Debugging)*, 2021. URL `https://arxiv.org/abs/2110.14297`.

[38] Wen Zhao, Jiawen Ding, Xueting Huang, and Yibo Zhang. Research on milling machine predictive maintenance based on machine learning and SHAP analysis in intelligent manufacturing environment. arXiv:2512.01205, 2025. Accepted at ICEIEC 2025.

[39] Yilun Zhou, Serena Booth, Marco Tulio Ribeiro, and Julie Shah. Do feature attribution methods correctly attribute features? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 9623–9633, 2022.